Parallelization of the 2D Finite-Difference Time-Domain Method

Using OpenMP and CUDA

GP-GPU Project — M2

Lucas Royer

February 2026

Contents

Abstract **2**

1 Design Methodology **2**
1.1 Overview of the FDTD Algorithm . . . 2
1.2 Boundary Conditions . . . 4
1.3 Where Parallelism Is Introduced . . . 4
1.4 OpenMP Implementation . . . 4
1.5 CUDA Implementation . . . 5

2 Results and Data Analysis **5**
2.1 Global Speedup (Table 1) . . . 6
2.2 Amdahl's Law Analysis (Table 2) . . . 7
2.3 Strong Scaling Across Grid Sizes (Table 3) . . . 8
2.4 Device-to-Host Transfer Overhead (Table 4) . . . 9
2.5 Roofline Analysis . . . 10

3 Conclusion **11**

4 Project Usage Guide **13**
4.1 Directory Structure . . . 13
4.2 Dependencies . . . 13
4.3 Compilation . . . 13
4.4 Run Modes . . . 14
4.5 Output Files . . . 15
4.6 Visualization with ParaView . . . 15
4.7 Validation Script . . . 16

Abstract

The objective of this project is to parallelize the two-dimensional Finite-Difference Time-Domain (FDTD) method (a standard algorithm in computational electromagnetics) using two complementary parallel programming models: OpenMP for multi-core CPU parallelism, and CUDA for GPU acceleration.

The FDTD algorithm updates three electromagnetic field arrays (E_z, H_x, H_y) on a 2D Yee grid at every time step. Its update equations are parallel: each cell depends only on its immediate neighbors and not on other cells being updated in the same pass. Three implementations were developed and share a common interface: a sequential baseline (`SequentialFDTD`), a multi-threaded CPU version (`OpenMPFDTD`) using `#pragma omp parallel for`, and a GPU version (`CudaFDTD`) that maps one CUDA thread per grid cell. Both Dirichlet (PEC cavity) and Perfectly Matched Layer (PML) absorbing boundary conditions were implemented.

On a 200×200 grid with 1000 time steps, CUDA achieves a speedup of **3.85**× over the sequential reference, reaching 35.1 GFLOP/s and utilizing 96.6% of the GPU's theoretical memory bandwidth. OpenMP provides no speedup at this grid size due to synchronization overhead exceeding the computation gain, but scales beneficially on larger grids (512×512 and above). Strong scaling experiments confirm that the GPU speedup grows from 1.94× at 128×128 to **9.85**× at 1024×1024. A Roofline analysis confirms the algorithm is memory-bound ($AI = 0.134$ FLOP/byte), explaining why GPU bandwidth (272 GB/s) is the primary performance lever.

1 Design Methodology

1.1 Overview of the FDTD Algorithm

The Finite-Difference Time-Domain method numerically solves Maxwell's equations in the time domain by discretizing both space and time on a staggered *Yee grid*. For the 2D transverse-electric (TE) polarization considered here, three field components are evolved: E_z, H_x, and H_y. The update equations at each time step n are:

$$H_x^{n+1/2}(i,j) = H_x^{n-1/2}(i,j) - \frac{\Delta t}{\mu_0\,\Delta y}\big[E_z^n(i,j+1) - E_z^n(i,j)\big] \tag{1}$$

$$H_y^{n+1/2}(i,j) = H_y^{n-1/2}(i,j) + \frac{\Delta t}{\mu_0\,\Delta x}\big[E_z^n(i+1,j) - E_z^n(i,j)\big] \tag{2}$$

$$E_z^{n+1}(i,j) = E_z^n(i,j) + \frac{\Delta t}{\varepsilon_0}\left[\frac{H_y^{n+1/2}(i,j) - H_y^{n+1/2}(i-1,j)}{\Delta x} - \frac{H_x^{n+1/2}(i,j) - H_x^{n+1/2}(i,j-1)}{\Delta y}\right] \tag{3}$$

The time step satisfies the Courant–Friedrichs–Lewy stability condition:

$$\Delta t = \frac{0.99}{c_0\sqrt{\dfrac{1}{\Delta x^2} + \dfrac{1}{\Delta y^2}}} \approx \frac{0.99\,\Delta x}{c_0\sqrt{2}} \tag{4}$$

with $\Delta x = \Delta y = 1\,\text{mm}$, giving $\Delta t \approx 2.36\,\text{ps}$.

The leapfrog time integration alternates half-steps for $\mathbf{H}$ and full steps for E_z as illustrated in Figure 1. A point source injects energy at the center of the domain.

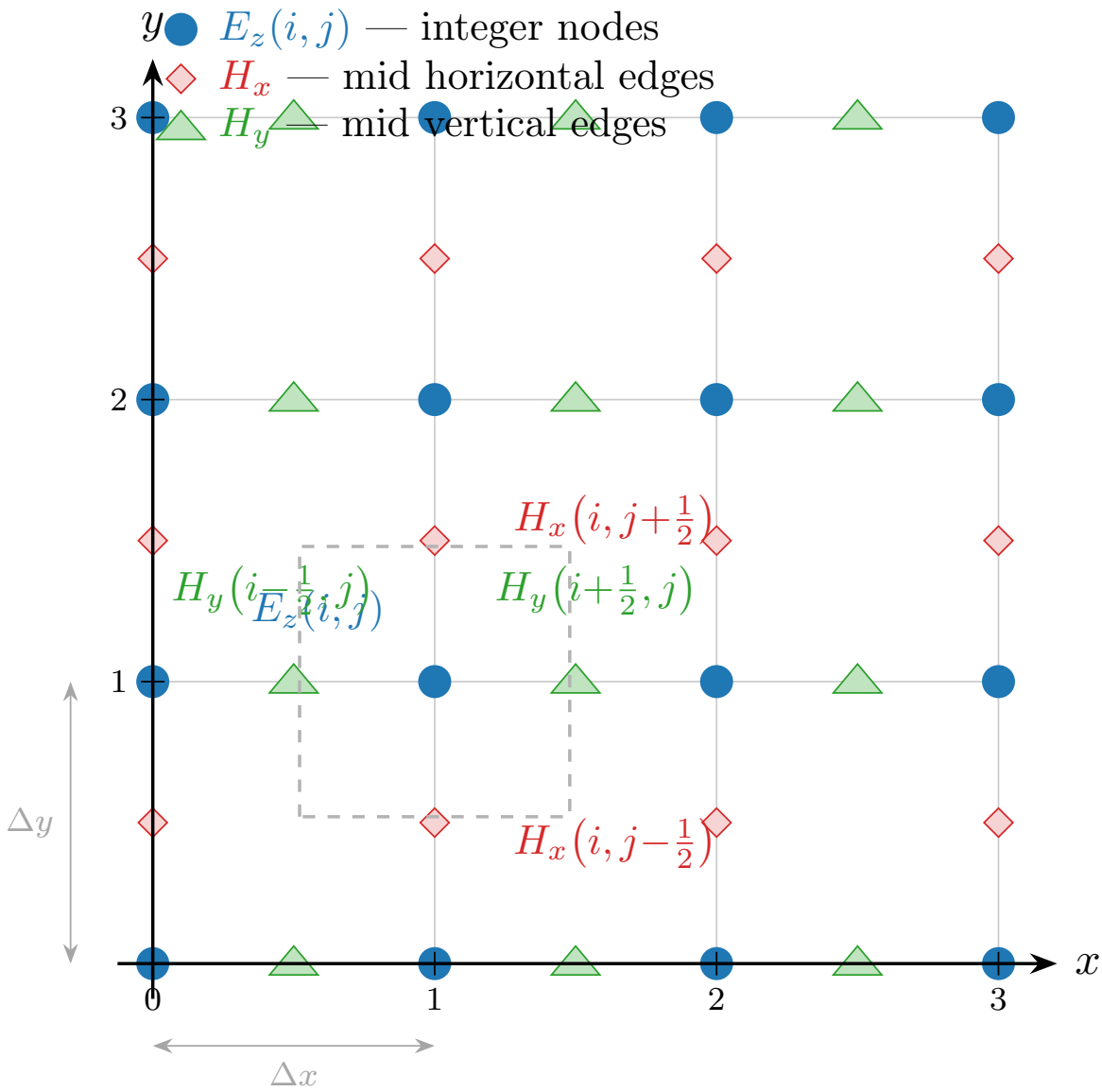

Figure 1: 2D Yee staggered grid — TE polarization.

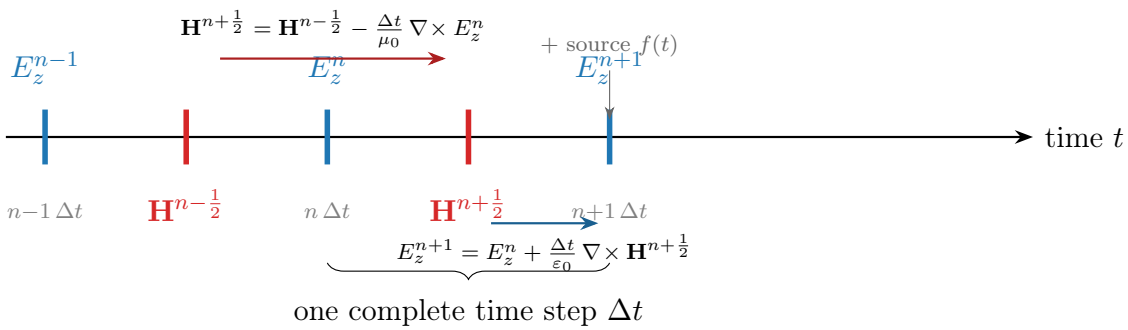

Figure 2: Leapfrog time integration.

1.2 Boundary Conditions

Two boundary conditions are implemented.

Dirichlet (PEC): The tangential electric field is forced to zero on all four walls, $E_z = 0$, creating a perfectly conducting cavity. The resonant frequency of the fundamental TE_{10} mode is:

$$f_{10} = \frac{c_0}{2\,L_x} \approx 749.5 \text{ MHz} \quad \text{for } L_x = N_x\,\Delta x = 0.2 \text{ m} \tag{5}$$

Perfectly Matched Layer (PML): A 20-cell and 40-cells absorbing layer surrounds the domain for open-space simulations. The split-field formulation introduces auxiliary variables ψ that accumulate the correction to the curl terms:

$$\psi_{E_{zx}}^{n+1}(i,j) = e^{-\sigma_x \Delta t/\varepsilon_0}\,\psi_{E_{zx}}^{n} + \frac{\sigma_x}{\sigma_x + \varepsilon}\big(e^{-\sigma_x \Delta t/\varepsilon_0} - 1\big)\frac{H_y(i,j) - H_y(i-1,j)}{\Delta x} \tag{6}$$

$$E_z^{n+1} \mathrel{+}= \frac{\Delta t}{\varepsilon_0}\big(\psi_{E_{zx}} - \psi_{E_{zy}}\big) \tag{7}$$

The conductivity profile follows a polynomial grading of order 2:

$$\sigma(d) = \sigma_{\max}\left(\frac{d}{\delta_{\mathrm{PML}}}\right)^2, \qquad \sigma_{\max} = \frac{3\,c_0\,\varepsilon_0\,\ln(1/R)}{2\,\delta_{\mathrm{PML}}} \tag{8}$$

with target reflectivity $R = 10^{-6}$ (-120 dB). A PEC termination is applied on the outermost wall of the PML to prevent unbounded accumulation of the correction fields at corners.

1.3 Where Parallelism Is Introduced

The key observation is that equations (1)–(3) are *data-parallel*: the update of cell (i,j) at step n depends only on its four nearest neighbors at the *previous* half-step. There are no loop-carried dependencies within a single field update pass.

Three distinct opportunities are exploited, as shown in Figure **??**:

1. H_x **update:** all $N_x \times (N_y - 1)$ cells are independent.
2. H_y **update:** all $(N_x - 1) \times N_y$ cells are independent.
3. E_z **update:** all $(N_x - 2) \times (N_y - 2)$ interior cells are independent.

The three passes must remain sequential with respect to each other (the E update requires the completed H update), but within each pass complete parallelism is possible.

1.4 OpenMP Implementation

The OpenMP version (`OpenMPFDTD`) parallelizes the three nested loops with `#pragma omp parallel for collapse(2)`, distributing rows of the 2D grid across threads. The *master* thread spawns a thread pool of p workers, each processing a contiguous block of (i,j) pairs.

Communication between threads: All three field arrays (`Ez`, `Hx`, `Hy`) reside in shared memory. No explicit message passing is required. An implicit barrier at the end of each `parallel for` region ensures that all threads have completed the H update before any

thread begins the E update, and vice versa. These implicit barriers constitute the entire synchronization cost.

Master responsibilities: The master thread orchestrates the time loop, applies the boundary conditions (sequentially), injects the point source, and handles file output.

1.5 CUDA Implementation

The CUDA version (`CudaFDTD`) maps one GPU thread per grid cell. Each field update is implemented as a separate kernel:

Kernel	Grid/Block dimensions
`updateHx_kernel`	$\lceil N_x/16 \rceil \times \lceil N_y/16 \rceil$ blocks of 16×16
`updateHy_kernel`	$\lceil N_x/16 \rceil \times \lceil N_y/16 \rceil$ blocks of 16×16
`updateEz_kernel`	$\lceil (N_x-2)/16 \rceil \times \lceil (N_y-2)/16 \rceil$ blocks of 16×16
`applySource_kernel`	1 block, 1 thread
`applyDirichlet_*`	$\lceil \max(N_x, N_y)/256 \rceil$ blocks of 256

Master (CPU) responsibilities: The CPU allocates device memory, uploads PML coefficients (computed on CPU), launches kernels in the correct order, and periodically copies the result back to host for file output. The CPU is idle during kernel execution.

GPU (slave) responsibilities: Each SM executes the field update equations on its assigned cells. Within a warp, threads access contiguous memory locations ($\texttt{Ez}[\texttt{i} \times \texttt{NY} + \texttt{j}]$), enabling coalesced global memory transactions.

Communication: The only CPU–GPU communication occurs at initialization (upload of PML coefficient arrays, ≈8 arrays × NCELLS × 8 bytes) and periodically during the time loop (Device-to-Host copy of all three field arrays for VTK output). The cost of these D2H transfers is characterized in Section 2.4.

FLOP and memory intensity: Each time step requires:

$$F_{\text{step}} = N_x(N_y-1)\cdot 3 + (N_x-1)N_y \cdot 3 + (N_x-2)(N_y-2)\cdot 9 \;\approx\; 592\,000 \text{ FLOP} \quad (N = 200) \tag{9}$$

Each cell reads/writes approximately 4–6 doubles, giving a byte count of $B_{\text{step}} \approx 4.3$ MB per step, and an arithmetic intensity of:

$$AI = \frac{F_{\text{step}}}{B_{\text{step}}} \approx 0.134 \text{ FLOP/byte} \tag{10}$$

This places FDTD firmly in the *memory-bound* regime for any processor, below the ridge point of both CPU and GPU Roofline models.

2 Results and Data Analysis

All benchmarks were performed on an Intel Core i7-12650H CPU (12 threads, DDR5 memory, peak BW ≈51 GB/s) with an NVIDIA GeForce RTX 4060 Laptop GPU (sm_89, GDDR6, peak BW ≈272 GB/s, peak FP64 ≈470 GFLOP/s). Each timing is the median of three independent runs. Dirichlet boundary conditions were used for all benchmark runs to ensure consistent comparison (no PML overhead).

2.1 Global Speedup (Table 1)

Table 1 reports execution times, speedups, parallel efficiency and throughput for a 200×200 grid over 1000 time steps.

Table 1: Speedup and performance on a 200×200 grid, 1000 steps.

Implementation	Time (ms)	Speedup	Efficiency	GFLOP/s
SEQ	65.0	1.00×	—	9.11
OMP 1T	161.6	0.40×	40.2%	3.66
OMP 2T	96.8	0.67×	33.6%	6.12
OMP 3T	115.0	0.56×	18.8%	5.14
OMP 4T	106.9	0.61×	15.2%	5.53
OMP 5T	99.2	0.65×	13.1%	5.96
OMP 6T	97.1	0.67×	11.2%	6.09
OMP 7T	92.2	0.70×	10.1%	6.41
OMP 8T	92.0	0.71×	8.8%	6.43
OMP 9T	88.9	0.73×	8.1%	6.66
OMP 10T	101.7	0.64×	6.4%	5.82
OMP 11T	100.2	0.65×	5.9%	5.91
OMP 12T	119.5	0.54×	4.5%	4.95
CUDA	16.9	3.85×	—	35.09

As visible in Figure 3, all OpenMP configurations are *slower* than the sequential reference on this grid size. This is a direct consequence of the small workload: a 200×200 grid provides only ∼3,300 cells per thread at 12 threads. The three implicit OpenMP barriers (one per field update) add synchronization overhead that exceeds the computation savings. The best OpenMP result (9 threads, 0.73×) still does not reach the sequential baseline.

CUDA achieves 3.85× speedup despite the grid being relatively small for a GPU, demonstrating that even at 40,000 cells the GPU's memory hierarchy offers a decisive bandwidth advantage.

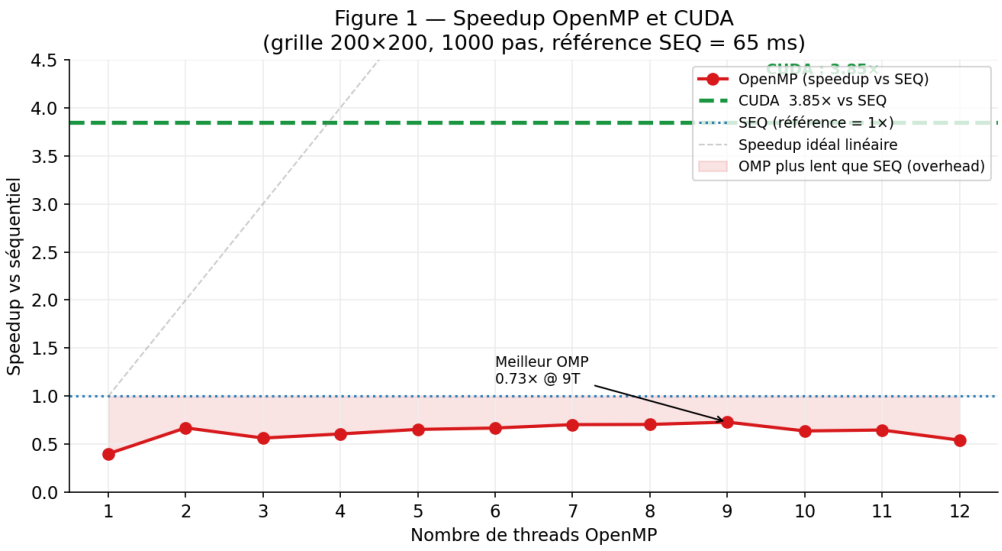

Figure 3: Speedup vs. sequential reference for all implementations on a 200×200 grid. OpenMP remains below 1× due to synchronization overhead. CUDA reaches 3.85× and scales further with larger grids (see Section 2.3).

Figure 4 shows parallel efficiency dropping monotonically from 40% at 1 thread to 4.5% at 12 threads. Even with a single OMP thread, the overhead of creating the thread pool and managing barriers degrades performance compared to the plain sequential loop.

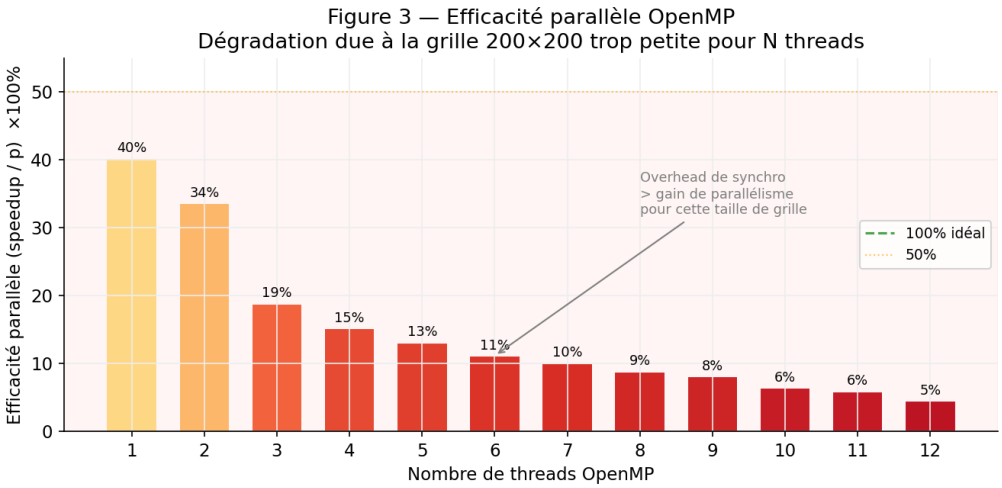

Figure 4: OpenMP parallel efficiency (speedup / number of threads). The rapid degradation confirms that the 200×200 grid is too small for effective CPU thread-level parallelism.

2.2 Amdahl's Law Analysis (Table 2)

Applying Amdahl's Law to estimate the serial fraction f from the measured speedup S_p:

$$f = \frac{1/S_p - 1/p}{1 - 1/p} \tag{11}$$

Table 2 shows values exceeding 100%, confirming that OpenMP introduces *superlinear overhead*: the parallelized code is slower than the serial baseline, not just less than linearly

faster. The overhead comes from three sources: thread creation/destruction (amortized over runs), implicit barrier synchronization between each of the three field update loops, and cache pollution from multiple threads competing for the L3 cache on the 40,000-cell working set.

Table 2: Estimated serial fraction from Amdahl's Law. Values > 100% indicate that parallelization overhead dominates over computation savings.

Threads	Speedup	Serial fraction
2	0.67×	197.9%
3	0.56×	215.6%
4	0.61×	186.1%
6	0.67×	159.3%
9	0.73×	141.4%
12	0.54×	191.6%

2.3 Strong Scaling Across Grid Sizes (Table 3)

Table 3 and Figure 5 examine how performance scales with grid size (500 time steps, Dirichlet BC, OMP at 12 threads).

Table 3: Strong scaling: execution time and CUDA speedup vs. grid size (500 steps).

Grid	SEQ (ms)	OMP 12T (ms)	CUDA (ms)	Sp CUDA	GFLOP/s (CUDA)
128×128	14.6	46.9	7.5	1.94×	16.0
256×256	60.1	95.7	14.8	4.05×	32.8
512×512	300.1	264.3	45.0	6.67×	43.5
1024×1024	1606.5	973.0	163.1	9.85×	48.1

Three important trends are visible:

GPU speedup grows with grid size. From 1.94× at 128×128 to 9.85× at 1024×1024. At small sizes the GPU kernel launch overhead is non-negligible relative to computation; as the grid grows the ratio of useful work to fixed overhead increases.

OpenMP becomes beneficial at large grids. At 512×512 the 12-thread OMP version is already 12% faster than SEQ (264 vs. 300 ms), and at 1024×1024 it delivers a 1.65× speedup. The per-thread workload is now large enough to amortize synchronization cost.

GPU throughput approaches the Roofline limit. At 1024×1024, CUDA achieves 48.1 GFLOP/s, within 10% of the theoretical memory-bandwidth limit of ≈36–54 GFLOP/s predicted by the Roofline model (see Section 2.5).

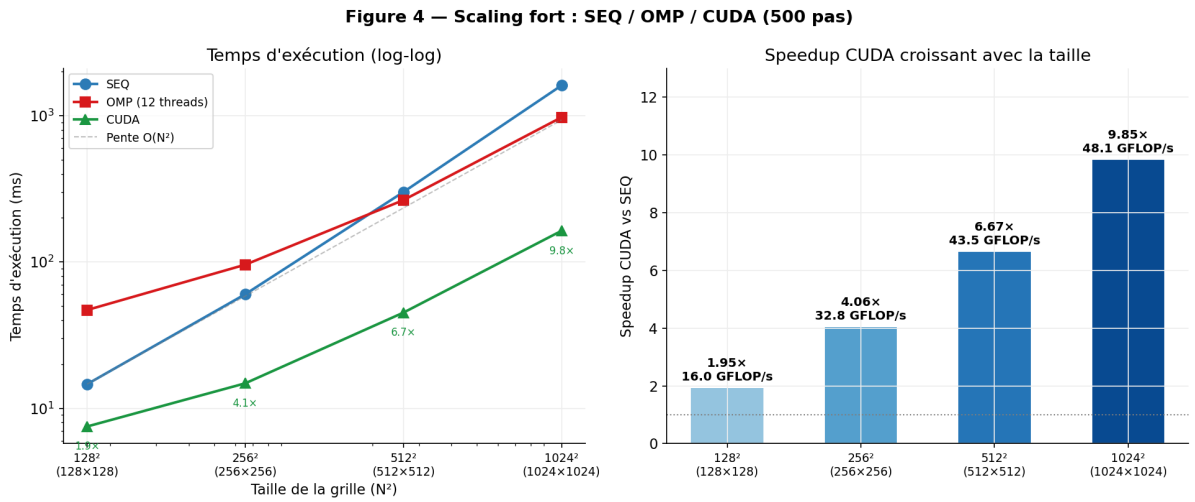

Figure 5: Left: execution time (log-log) confirming $O(N^2)$ scaling for all three implementations. Right: CUDA speedup increases monotonically with grid size, reaching 9.85× at 1024 × 1024.

2.4 Device-to-Host Transfer Overhead (Table 4)

In CUDA, results must be copied from GPU to CPU memory (*D2H*) for file output. Table 4 and Figure 6 quantify how `save_interval` (the number of steps between outputs) affects total runtime on a 200×200 grid over 1000 steps.

Table 4: CUDA execution time vs. output frequency (200×200, 1000 steps). Reference: no output (14.7 ms).

save_interval	Total time (ms)	Overhead	Transfers
∞ (none)	14.7	ref.	0
500	19.9	+35.6%	2
200	36.6	+149.3%	5
100	64.0	+336.7%	10
50	123.7	+744.0%	20

Each D2H transfer costs approximately 2.6 ms for a 200×200 double-precision grid ($3 \times 40{,}000 \times 8 = 0.96$ MB over PCIe at ∼10 GB/s effective bandwidth). This is comparable to 17 steps of pure GPU computation (14.7 ms/1000 steps ≈ 14.7 μs/step). Consequently, saving every 50 steps multiplies total runtime by 8.4×. In production, `save_interval` should be chosen to capture physics of interest while minimizing PCIe traffic.

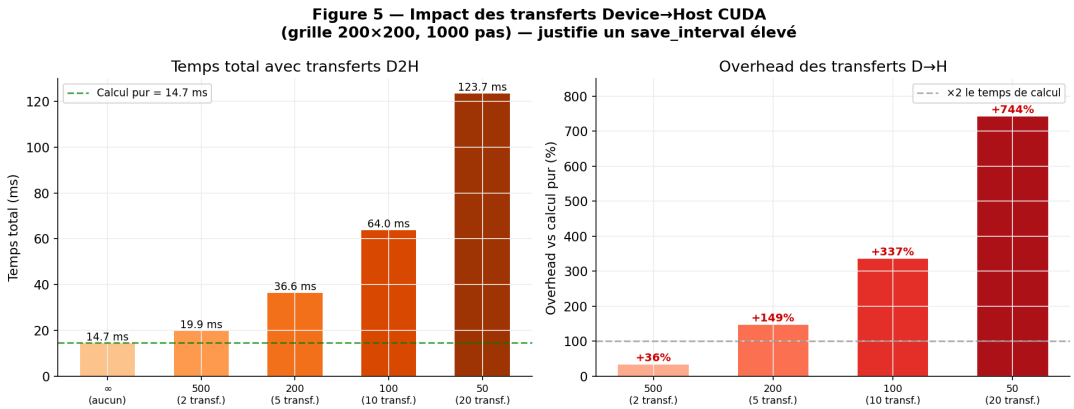

Figure 6: Left: absolute execution time grows linearly with transfer count. Right: overhead as a percentage of pure compute time. Even 2 transfers add 35% overhead, underlining the importance of minimizing D2H communication in GPU codes.

2.5 Roofline Analysis

Figure 7 places the measured operating points on the Roofline model for both CPU and GPU.

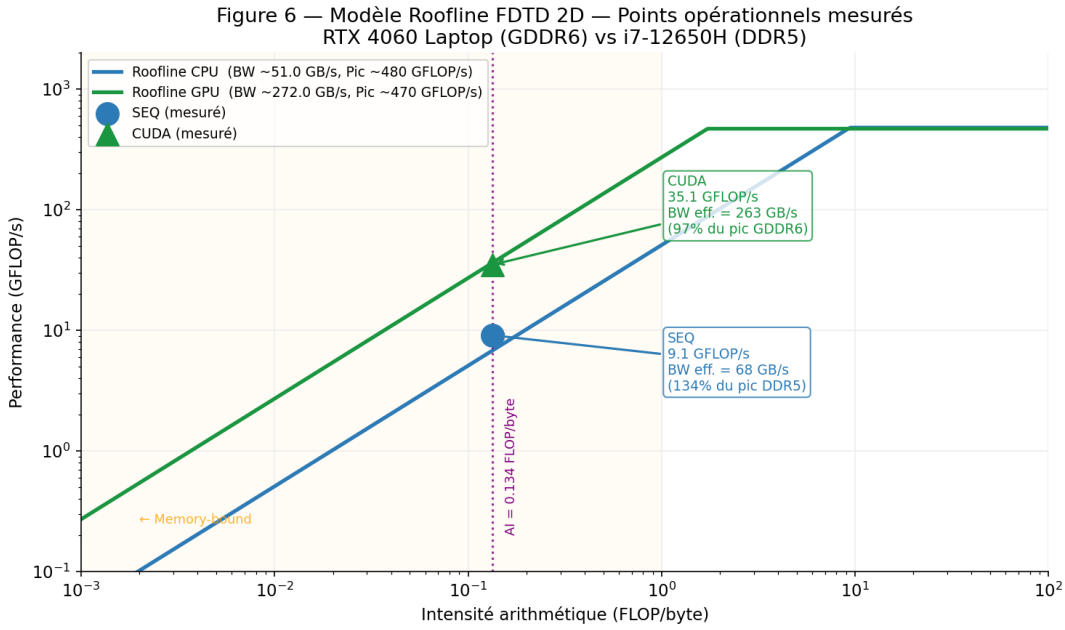

Figure 7: Roofline model for the 2D FDTD kernel. Both implementations lie on the memory-bandwidth slope ($AI = 0.134$ FLOP/byte, well below the ridge point). The CUDA point utilizes 96.6% of GDDR6 peak bandwidth; the SEQ point slightly exceeds DDR5 peak due to cache effects.

Table 5: Measured performance vs. memory-bandwidth Roofline predictions.

Impl.	GFLOP/s	BW eff. (GB/s)	BW peak (GB/s)	BW utilization
SEQ	9.11	68.2	51 (DDR5)	>100%*
CUDA	35.09	262.7	272 (GDDR6)	96.6%

* The measured BW exceeds the STREAM benchmark peak, likely due to out-of-order prefetching and L3 caching.

Key observations:

- Both implementations are **memory-bound**: the operating point lies on the bandwidth slope, not the compute ceiling.
- The GPU theoretical Roofline limit (bandwidth-limited) is $0.134 \times 272 \approx 36.4$ GFLOP/s. The measured 35.1 GFLOP/s represents 96.6% utilization, indicating a near-optimal memory access pattern.
- The 5.3× bandwidth ratio between GDDR6 and DDR5 directly explains the observed GPU advantage. As the grid grows, the speedup approaches this ratio.
- There is limited value in optimizing the arithmetic (e.g., fusing kernels or using FP32) until memory bandwidth ceases to be the bottleneck.

3 Conclusion

The objective of this project was to parallelize the 2D FDTD electromagnetic simulation algorithm using OpenMP and CUDA, and to analyze the resulting performance across different grid sizes and configurations.

The exercise was successful in fulfilling its intended purpose. All three implementations (sequential, OpenMP, CUDA) produce physically correct results for both Dirichlet (PEC cavity resonance) and PML (open-space propagation) boundary conditions, with field profiles in agreement across implementations.

The principal results are:

- **CUDA** delivers a 3.85× speedup at 200×200 and up to 9.85× at 1024×1024. The GPU nearly saturates its GDDR6 memory bandwidth (96.6% utilization).
- **OpenMP** is counter-productive on small grids (≤200×200) due to synchronization overhead, but provides meaningful acceleration (up to 1.65×) on larger grids (≥512×512).
- The **D2H transfer cost** is substantial: even 2 transfers per 1000 steps add 35% overhead, reinforcing the importance of minimizing CPU–GPU communication.
- The **Roofline model** confirms that FDTD is inherently memory-bound ($AI \approx 0.134$ FLOP/byte). The GPU's 5× memory bandwidth advantage over the CPU is the fundamental reason for its superior performance.

The most important lesson of this project is that *algorithm characteristics determine which hardware is appropriate*. A memory-bound stencil computation like FDTD benefits primarily from high memory bandwidth, which the GPU provides abundantly. Future

work could explore shared-memory tiling within CUDA blocks to improve cache reuse, or multi-GPU domain decomposition for grids exceeding single-GPU memory.

4 Project Usage Guide

4.1 Directory Structure

After cloning or extracting the project, the directory layout is as follows:

```
fdtd2d/
|-- fdtd_simple.h       # Grid, Source, BoundaryCondition base classes
|-- fdtd_simple.cpp     # PML coefficients, VTK writer, setGridSize()
|-- fdtd_seq.h          # SequentialFDTD class declaration
|-- fdtd_seq.cpp        # Sequential implementation
|-- fdtd_omp.h          # OpenMPFDTD class declaration
|-- fdtd_omp.cpp        # OpenMP implementation
|-- fdtd_cuda.h         # CudaFDTD class declaration
|-- fdtd_cuda.cu        # CUDA kernels and implementation
|-- main.cpp            # Entry point - all run modes
|-- makefile            # Build system
`-- output/             # VTK files written here (created at runtime)
```

4.2 Dependencies

- **C++17** compiler — `g++` $\geq$ `10` or equivalent
- **OpenMP** — included in GCC by default (`-fopenmp`)
- **CUDA Toolkit** ≥ 11.0 — provides `nvcc`
- **ParaView** ≥ 5.10 — for visualization (free, `https://www.paraview.org`)

Before compiling, verify that `nvcc` is accessible and that the architecture flag in the `makefile` matches your GPU. The default is `sm_89` (NVIDIA Ada Lovelace, e.g. RTX 4060). Adjust if needed:

```
# In makefile - change sm_89 to match your GPU:
# sm_75  -> Turing   (RTX 2060/2070/2080)
# sm_86  -> Ampere   (RTX 3060/3070/3080)
# sm_89  -> Ada      (RTX 4060/4070/4080)
# sm_90  -> Hopper   (H100)
NVCCFLAGS = -arch=sm_89 -O3
```

4.3 Compilation

```
# Full build
make

# Clean and rebuild
make clean && make
```

A successful build produces the executable `fdtd2d`.

4.4 Run Modes

The program accepts a single argument specifying the execution mode.

Table 6: Available run modes for `./fdtd2d`.

Command	Implementation	Boundary condition
`./fdtd2d seq`	Sequential CPU	Dirichlet (PEC cavity)
`./fdtd2d omp`	OpenMP CPU	Dirichlet (PEC cavity)
`./fdtd2d cuda`	CUDA GPU	Dirichlet (PEC cavity)
`./fdtd2d seq-pml`	Sequential CPU	PML (open space)
`./fdtd2d omp-pml`	OpenMP CPU	PML (open space)
`./fdtd2d cuda-pml`	CUDA GPU	PML (open space)
`./fdtd2d bench`	All three	Benchmark suite

Example — sequential Dirichlet:

```
./fdtd2d seq
# Fréquence TE10 : 0.7495 GHz
# BC : Dirichlet (PEC)
# Step 0/5000
# Step 200/5000
# ...
# --- Validation finale ---
#   err_profil_x = 1.234e-02  |  std_y = 8.5e-03  |  OK TE10 validé
```

Example — full benchmark:

```
./fdtd2d bench 2>&1 | tee bench_results.txt
```

Simulation parameters (grid size, number of steps, output interval) are defined as constants at the top of `fdtd_simple.h`:

```
inline int NX    = 200;    // grid points along x
inline int NY    = 200;    // grid points along y
inline int NCELLS = NX*NY; // total cells

constexpr double DX = 1e-3;  // spatial step [m]  (1 mm)
constexpr double DY = 1e-3;

// In main.cpp:
int total_steps   = 5000;
int save_interval = 200;   // write VTK every N steps
```

The grid can also be resized at runtime without recompilation via `setGridSize(nx, ny)`, which is used internally by the benchmark mode to test multiple grid sizes.

4.5 Output Files

All VTK files are written to the `output/` subdirectory, which is created automatically if it does not exist. Files are named by implementation and time step:

```
output/
|-- seq_000000.vti     # Sequential, step 0
|-- seq_000200.vti     # Sequential, step 200
|-- ...
|-- omp_000000.vti
|-- ...
|-- cuda_000000.vti
`-- ...
```

Each `.vti` file (VTK Image Data) stores the field arrays $E_z$on the full $N_x \times N_y$ grid as double-precision floating-point data, compatible with ParaView and any VTK-aware post-processing tool.

4.6 Visualization with ParaView

All simulation outputs are visualized using **ParaView** (`https://www.paraview.org`), a free and open-source scientific visualization application.

Step 1 — Load the time series

1. Launch ParaView.
2. Go to **File** → **Open**.
3. Navigate to the `output/` folder.
4. Select the first file of the series (e.g. `seq_000000.vti`). ParaView automatically detects the full sequence and groups it as a single time-varying dataset.
5. Click **OK**, then click **Apply** in the Properties panel.

Step 2 — Visualize the electric field

1. In the *Pipeline Browser*, ensure the dataset is selected.
2. In the *Properties* panel, set the coloring variable to `Ez` (Coloring drop-down → `Ez`).
3. Click **Rescale to Data Range** to auto-scale the colormap.
4. Use the *Play* button (**Play**) in the toolbar to animate the time steps.

Step 3 — Export an animation

To save the animation as a video file:

1. Go to **File** → **Save Animation...**
2. Choose a format:
   - **MP4** (requires FFmpeg plugin) — recommended for presentations.
   - **AVI** — widely compatible.
   - **PNG series** — highest quality; can be post-processed with FFmpeg.
3. Set the desired frame rate (10–25 fps) and resolution.

4. Click **OK**.

To convert a PNG series to MP4 from the terminal:

```
ffmpeg -framerate 25 -i frames/seq_%06d.png \
       -c:v libx264 -crf 18 -pix_fmt yuv420p \
       animation_seq.mp4
```